

Note Méthodologique : Stratégie de Dédoublonnage des Listes de Codes (DDI 3.3/6.0)

Agent de Data Science — SSP Cloud / Onyxia

2026-07-21

Introduction

Cette note décrit la méthodologie implémentée dans le pipeline `metadata-scrubber` pour identifier les listes de codes (`CodeList`) redondantes au sein d'un fichier XML DDI (Data Dictionary Interface).

Scénario : un fichier de métadonnées peut contenir plusieurs définitions de `CodeList` qui, bien que nommées différemment ou légèrement variées, décrivent le même concept.

Référence : le fichier **BTS.xml** (réf. 4,7 Mo) — 156 `CodeLists`, 831 catégories, 503 références `Variable` → `CodeList`, 54 `CodeLists` redondantes dans 29 groupes.

1. Modèle de Données DDI et Extraction

1.1. Structure du fichier DDI

Un fichier DDI 3.3 est une suite d'objets enfermés dans des balises `<Fragment>` (~2 573 pour `BTS.xml`). Chaque `Fragment` contient **exactement un objet** :

- **CodeList** : la liste de codes elle-même
- **Category** : le libellé d'une valeur de code
- **Variable** / **RepresentedVariable** : référence une `CodeList`

Ordre arbitraire : les `CodeList` peuvent apparaître avant les `Category` correspondantes → approche en **deux passes** obligatoire.

Structure XML : `<Code>`

Chaque `<Code>` ne contient pas le libellé directement :

```
<Code>
  <r:ID>4dd22a77-9078-4417-9710-48699af3298a</r:ID>
  <r:CategoryReference>
    <r:ID>d72b4a6c-b668-46c9-98d9-7ba4663e7fa0</r:ID>
  </r:CategoryReference>
  <r:Value>4</r:Value>
</Code>
```

- r:Value : valeur brute
- r:CategoryReference > r:ID : référence vers une Category

Structure XML : <Category>

Le libellé est stocké séparément :

```
<Category>
  <r:ID>d72b4a6c-b668-46c9-98d9-7ba4663e7fa0</r:ID>
  <r:Label><r:Content xml:lang="fr-FR">Autres organismes publics</r:Content></r:Label>
  <CategoryName>domempl-4</CategoryName>
</Category>
```

Structure XML : <Variable> référençant une CodeList

```
<Variable>
  <r:ID>7686be9c-7849-4941-bccc-62206f764a12</r:ID>
  <VariableName><r:String xml:lang="fr-FR">domempl</r:String></VariableName>
  <r:Label><r:Content xml:lang="fr-FR">Domaine d'emploi</r:Content></r:Label>
  <VariableRepresentation>
    <r:CodeRepresentation>
      <r:CodeListReference>
        <r:ID>36a67d27-866b-40c6-9e4d-286d1f3d31d1</r:ID>
      </r:CodeListReference>
    </r:CodeRepresentation>
  </VariableRepresentation>
</Variable>
```

1.2. Pipeline d'extraction (4 phases)

1. Lecture XML (lxml, namespaces ignorés)
filtrer par local(frag) == "Fragment"
2. Passe 1 - Indexer les Catégories
{r:ID → r:Label > r:Content.text}
Résultat : 831 cat_id → libellé

3. Passe 2 - Extraire les CodeLists
 Pour chaque <Code> : extraire Value + CategoryRef
 Résoudre libellé via index
 Résultat : 156 codelists avec codes résolus
4. Passe 3 - Extraire les références Variables
 Chercher CodeListReference au plus profond
 Résultat : 503 références Variable → CodeList

2. L'Entonnoir de Détection

Principe : un entonnoir en trois niveaux, du plus simple au plus coûteux.

2.1. Phase 1 — Doublons Exact

Méthode :

1. Normalisation : Unicode NFC, minuscule, espaces multiples → unique
2. Signature : tuple **trié** des paires (**valeur**, **libellé**) normalisées
3. Regroupement par signature identique

Nom et identifiant volontairement exclus de la signature →
 deux listes identiques nommées différemment sont détectées.

Exemple : 18 listes à 1 code

18 CodeLists nommées différentes (N-categorie_juridique, N-region, N-conv_coll, N_DEpartements, ...).

Chacune contient un seul code :

Valeur	Libellé
cf	Voir dictionnaire des nomenclatures

Toutes 18 partagent **exactement la même signature** → doublons exacts.
 Probablement des **codes génériques** « consulter le dictionnaire » dupliqués
 par l'outil de modélisation (Collectica).

Caractéristiques

Aspect	Détail
Complexité	$O(N \cdot C)$ — linéaire, N = nb CodeLists, C = nb codes moyen

Aspect	Détail
Réseau	Aucun appel nécessaire
Précision	100 % — zéro faux-positif
Limite	1 différence dans une liste de 10 suffit pour passer

Sur BTS.xml : **29 groupes** détectés, **54** CodeLists redondantes.

2.2. Phase 1 bis — Quasi-Doublons (Similarité Floue)

Méthode :

1. À partir des uniques (une par groupe exact $\rightarrow$ ~102 pour BTS.xml), concaténer nom + paires `valeur:libellé` en une chaîne
2. Comparer **toutes les paires** avec `difflib.SequenceMatcher`
3. Seuils : **0.90** = détecté ; **0.80** = flagged pour inspection

Exemples BTS.xml

Paire	Score	Différence
SOURCE SOURCE_1	0.9985	Identique (casse près)
N_tr_ca_23 TR_CA	0.9896	Majuscules/espace sur 2 codes
ETAT_ET ETAT_ET_1	0.9961	Quasi-identique
N_domempl_23 DOMEMPL	0.86	NON détectée (accent + ordre)

Caractéristiques

Aspect	Détail
Complexité	$O(U^2 \cdot L)$, U 102 uniques
Faux négatifs	Différence $>$ ~10 % du texte
Faux positifs	Deux listes de plages de valeurs similaires

2.3. Phase 2 — Doublons Sémantiques (Embeddings + LLM)

Les deux phases précédentes ne comprennent **pas le sens** : possible de “Sexe” vs “Genre” avec codes différents.

2.3.1. Embeddings de CodeLists

1. Vectorisation : modèle `qwen3-embedding-8b` sur nom + label + codes
2. Cosinus entre toutes les paires (102 vecteurs $\rightarrow$ 102×102)
3. Filtrage : **0.90**

4. **Juge LLM** : gemma4-26b-moe sur les meilleures candidates

```
{
  "meme_concept": true,
  "confiance": 0.95,
  "raison": "Mêmes codes départementaux français ; seule la casse et les accents diffèrent."
}
```

2.3.2. Embeddings de Variables (usage)

Approche inversée : on compare les **variables** qui référencent les CodeLists.

Liste	Variables	var_sig
N_domempl_28	domempl, domempl_empl	("domempl", "domempl_empl")
DOMEMPL	domempl, domempl_empl, DOMEMPL, DOMEMPL_EMPL	("DOMEMPL", "DOMEMPL_EMPL", "domempl", "domempl_empl")

Même sémantique des 2 premières variables → mêmes concept métier. Particulièrement efficace quand toutes les CodeLists sont référencées (BTS.xml : 156/156).

Caractéristiques

Aspect	Détail
Complexité	$O(U^2 \cdot D)$ pour embeddings, $D = 8192$
Coût	Appels réseau ; dépend de OPENAI_BASE_URL + OPENAI_API_KEY
Précision	La plus élevée (validation LLM)
Limite	Coût calcul, dépendance réseau

3. **Signaux de Renforcement : var_sig**

Une CodeList est **toujours** référencée par au moins une variable. Deux listes utilisées par les **mêmes variables** sont quasi-certainement identiques.

```
var_sig(CL) = tuple(sorted( noms_des_variables_qui_la_referencent ))
```

3.1. Types de recoupement

Signal	Signification
même usage	Ensembles de variables identiques → fort indicateur de redondance
~ usage partiel	Intersection significative → variation contextuelle possible
usages disjoints	Aucune variable commune → probablement distinctes ou besoin de consolidation

3.2. Exemples BTS.xml

Groupe A : var_sig identique + contenu identique → doublon exact confirmé

CodeList	var_sig
N_saison_23	("saison",)
N_SAISON_Validité 2022	("saison",)

Groupe B : var_sig identique + contenu différent → à vérifier

CodeList	var_sig
N_champ_23	("champ",)
N_CHAMP_2022	("champ",)

Groupe C : var_sig disjoints + contenu identique → à consolider

CodeList	var_sig	Codes
SOURCE	("source",)	6 codes identiques
SOURCE_1	("source_1",)	6 codes identiques

4. Le Flag --registry : Dédoublonnage Transversal Itératif

4.1. Pourquoi --registry ?

Un seul run du pipeline identifie les redondances **au sein d'un unique fichier traité**. Mais les mêmes CodeLists peuvent apparaître dans plusieurs opérations (BTS, BPE, CPE...). Sans registre partagé, chaque nouvel run redécouvre les paires déjà validées dans les opérations précédentes.

Le registre `cleaned_codelists.json` est un **catalogue transversal de vérité** : il s'alimente au fil des opérations et alimente la détection du run suivant, y compris les appariements **cross-opération**.

4.2. Flux transversal

```
Opération 1 : BTS_demo.xml (sans --registry)
  → codelist_duplicates.json → 4 paires détectées (exact + flou + sémantique)
  → Interface web : examiner / approuver / rejeter
  → /api/registry/save → cleaned_codelists.json (mères BTS)
```

```
Opération 2 : BPE_demo.xml avec --registry cleaned_codelists.json
  → masters BTS injectés dans la détection
  → CodeLists BTS doublons exclues (déjà validées)
  → codelist_duplicates.json → paires BPE intra + BPE BTS (cross-fichier)
  → /api/registry/save → cleaned_codelists.json mis à jour (BTS + BPE)
```

```
Opération N : fichier suivant avec --registry cleaned_codelists.json
  → registry = tous les masters validés précédemment
  → détection = intra-fichier + cross-opération sur les masters cumulés
  → registre s'enrichit, les paires résiduelles diminuent à chaque étape
```

Plus le registre grossit, plus la détection évite les réitérations et capture les appariements inter-opérations.

4.3. Ce que fait `--registry` dans le pipeline (`main.py`, §2 bis)

Lors du lancement avec `--registry cleaned_codelists.json` :

1. **Lecture** du registre nettoyé (schéma v2, migration automatique v1 → v2)
2. **Injection** des entrées comme **masters prioritaires** (prépendé à la liste des CodeLists)
3. **Exclusion** des CodeLists dont l'ID figure dans `replaces[].id` (déjà traitées)
4. **Continuation** : la détection exacte/floue/sémantique ne voit plus que les nouveautés

```
# Simplifié depuis main.py (lignes 141-155) :
registry_cls, replaced_ids = load_cleaned_codelists(registry_path)
codelists = [
    cl for cl in codelists
    if cl.id not in registry_ids and cl.id not in replaced_ids
]
codelists = registry_cls + codelists # masters d'abord
```

4.4. Schéma du registre nettoyé (v2)

```
{
  "version": 2,
  "generated_at": "2025-xx-xxTxx:xx:xx+00:00",
  "source_registry": "s3://.../codelist_duplicates.json",
  "codelists": {
    "uuid-master": {
      "id": "uuid-master",
      "name": "Nom_master",
      "label": "Libellé master",
      "codes": [["1", "Valeur 1"], ["2", "Valeur 2"]],
      "codes_count": 2,
      "vars": ["var1", "var2"],
      "replaces": [
        {
          "id": "uuid-dup",
          "name": "Nom_dup",
          "detection_types": ["exact", "usage"],
          "confidence": 1.0
        }
      ],
      "first_added_at": "2025-xx-xx...",
      "updated_at": "2025-xx-xx..."
    }
  }
}
```

4.5. Sync incrémental (sync_cleaned_registry)

Lors de `/api/registry/save` :

- **Décision approuvée** → ajout au `replaces` du parent (création de l'entrée parente si inexistante)
 - **Décision dé-approuvée** → retrait du `replaces` (uniquement si l'ID existe toujours dans les duplicates du parent courant)
 - **replaces vide** → entrée supprimée **uniquement** si c'est le sync qui l'a vidée (les entrées ajoutées à la main survivent)
 - **Name/label/codes/vars** ne sont **JAMAIS** modifiés par le sync (éditions manuelles préservées)
-

5. Walkthrough : Exemples avec les Datasets Demo

Le dossier `demo/` contient des fichiers XML de petite taille, générés depuis les sources S3, avec des paires connues pour illustrer et tester le pipeline.

Générer les fichiers : `uv run demo/make_demo_files.py`.

5.1. `demo/BTS_demo.xml` — Run 1 (sans registry)

Contexte : 10 CodeLists issues de `BTS.xml` : 4 paires connues + 2 listes uniques.

Paire 1 — Exact : `N_NAF_6` `N-ACTIVITE-FRANCAISE-6-POSTES`

Valeur	N_NAF_6	N-ACTIVITE-FRANCAISE-6-POSTES
AZ	Agriculture, sylviculture et pêche	Agriculture, sylviculture et pêche
BE	Industrie manufacturière, ind. extractives et autres	Industrie manufacturière, ind. extractives et autres
FZ	Construction	Construction
GI	Commerce de gros et de détail, transports, hébergement et restauration	Commerce de gros et de détail, transports, hébergement et restauration
JU	Services divers	Services divers
OQ	Administration publique, enseignement, santé et action sociale	Administration publique, enseignement, santé et action sociale
Vide	Non renseigné	Non renseigné

Détection : exact (signature (`valeur`, `libellé`) identique sur les 7 codes).

Signal d'usage : mêmes variables référentes dans les deux listes. **Décision expert** : `N_NAF_6` retenue comme maître → `N-ACTIVITE-FRANCAISE-6-POSTES` placée dans `replaces[]` de `N_NAF_6`.

Paire 2 — Fuzzy : `N_nat_etab_23` `NAT_ETAB`

3 codes partagés (CCP, COM, STD) — libellés identiques mais noms de CodeList différents et description légèrement différente.

Détection : `SequenceMatcher` 0.90.

Paire 3 — Fuzzy : N_champ_23 N_CHAMP_2022

5 codes partagés (DADS, DADS-PUBLIC, DSN, DSN-PUBLIC, PE).

Paire 4 — Sémantique (LLM) : N_etat_23 Etat_2020

Pas de chevauchement direct de codes — détectée uniquement par embeddings + juge LLM.

Résultat run 1 : `codelist_duplicates.json` contient 4 paires. Sauvegarde dans `cleaned_codelists.json` (v1) : 4 entrées de maître avec leurs `replaces[]`.

5.2. demo/BPE_demo.xml — Run 2 (avec `--registry cleaned_codelists.json`)

Contexte : 9 CodeLists issues de BPE.xml — 6 listes BPE natives, 2 copies de BTS avec IDs neufs, 1 unique à ajouter manuellement.

Paires intra-BPE (indépendantes du registre)

Paire 5 — L_VAR_SPECIF_2023 L_VAR_SPECIF_2021

Même concept, variantes annuelles différentes (codes 0/1/_U/_Z pour 2023 vs OUI/NON/IND/NC pour 2021).

Paire 6 — L_AN_BPE_EVOL_2023 L_AN_BPE_EVOL_2022

Codes : 2013/2017/2018/2023 pour 2023 vs 2017/2022 pour 2022 — même concept, années différentes.

Paires cross-fichier (détectées GRÂCE AU REGISTRY)

Paire 7 — NAF_6_POSTES_BPE registre N_NAF_6 (exact, cross-fichier)

NAF_6_POSTES_BPE est une copie exacte de N_NAF_6 avec des UUIDs entièrement neufs. Sans le registre, le pipeline ne détecterait qu'une similarité de contenu brute. Avec le registre, N_NAF_6 est injectée comme **master prioritaire** et servira de candidat à la comparaison — la détection exacte la reconnaît immédiatement.

Paire 8 — ETAT_UNITE_BPE registre N_etat_23 (fuzzy, cross-fichier)

ETAT_UNITE_BPE est une copie de N_etat_23 avec des tweaks sur les libellés (ex. **Unité active** → **Unité en activité**, **Non renseigné** → **Non renseignée**). La détection par signature exacte ne la capturera pas, mais la signature floue (`SequenceMatcher 0.90`) plus le registre qui place N_etat_23 comme master prioritaire permettent sa détection.

5.3. Registre après opération 2 (BPE)

`cleaned_codelists.json` (v2) contient maintenant 6 entrées de maître :

Maître	Origine	<code>replaces[]</code> contient
N_NAF_6	BTS_demo	N-ACTIVITE-FRANCAISE-6-POSTES (run 1) + NAF_6-POSTES_BPE (run 2)
N_nat_etab_23	BTS_demo	NAT_ETAB (run 1)
N_champ_23	BTS_demo	N_CHAMP_2022 (run 1)
N_etat_23	BTS_demo	Etat_2020 (run 1) + ETAT_UNITE_BPE (run 2)
L_VAR_SPECIF_2023	BPE_demo	L_VAR_SPECIF_2021 (run 2)
L_AN_BPE_EVOL_2023	BPE_demo	L_AN_BPE_EVOL_2022 (run 2)

5.4. Ajout manuel : L_QUALITE_ZONAGE_2023

Cette CodeList est unique (aucun doublon intra ou cross-fichier détecté). L'expert peut l'ajouter **manuellement** au registre (bouton "Ajouter au registre" dans l'interface), sans validation de paire. Elle sera présente comme master dans le registre pour les opérations futures.

6. Tableau Récapitulatif — Dédoublonnage BTS.xml

Méthode	Seuil	Complexité	Output BTS.xml (156 CLs)
Signature exacte	—	$O(N \cdot C)$	29 groupes, 54 CLs redondantes
Similarité floue	0.90	$O(U^2 \cdot L)$	~10 paires
Embeddings de liste	0.90 + LLM	$O(U^2 \cdot D)$	Dépend de config OpenAI
Embeddings de variable	0.92	$O(V^2 \cdot D)$	Dépend de config OpenAI
var_sig	Identique	$O(N)$	17 groupes de même usage

- N = nb CodeLists (156), C = nb codes moyen (10)
 - U = nb uniques après dédup exact (102)
 - V = nb variables (287), D = dimension embeddings (8192)
-

7. Perspectives

7.1. Performance à Grande Échelle

- **MSA / MinHash** pour remplacer la comparaison par paires $O(n^2)$
- **Parallélisation** du pipeline sur Kubernetes via Argo Workflows

7.2. Robustesse

- Détection de **sous-ensembles** : une CodeList peut être incluse dans une autre
- Embeddings spécialisés sur le vocabulaire statistique Insee

7.3. Pipeline Reproducible

- Transformation vers pipeline **targets** (R) ou Python équivalent
- Intégration **CI/CD** : déclencher le dédoublement à chaque commit

7.4. Analyse Fine de l'Usage

- Hiérarchie des variables (groupes, familles d'enquêtes)
 - Pondération par importance (métrique de centralité)
 - Contextes d'enquête : une CodeList valide dans un contexte mais pas un autre
-

Conclusion

Un entonnoir en trois niveaux — **exact** → **flou** → **sémantique** — complété par un signal d'usage (**var_sig**) et un registre transversal itératif (**--registry**) permet de couvrir toute la palette des redondances : copies conformes, variantes mineures, synonymes conceptuels, **et doublons cross-opération**.

Les datasets **demo/** illustrent ce workflow complet :

1. **BTS_demo** (run 1) : 4 paires intra-fichier détectées (exact, fuzzy, sémantique)
2. **BPE_demo** (run 2, avec registre) : 2 paires intra-BPE + 2 paires cross-fichier (NAF_6_POSTES_BPE N_NAF_6, ETAT_UNITE_BPE N_etat_23)

3. Le registre `cleaned_codelists.json` accumule 6 maîtres validés couvrant les deux opérations